Guía de revisión técnica

Compilador Nova para la arquitectura VLIW CERBERO · CE1108 Compiladores e Intérpretes · Proyecto II

Cada sección tiene dos partes: la **explicación**, que es lo que hay que poder contar en voz alta, y una **tabla de referencias** con el archivo y las líneas donde vive ese código, para abrirlo en el momento si lo piden. Los recuadros azules son el punto que conviene no omitir de cada tema.

1. Tabla de símbolos

La tabla es una **pila de ámbitos con búsqueda hacia afuera**. Al entrar a un bloque se apila un ámbito y al salir se desapila; `symtab_lookup` busca desde el ámbito actual hacia el global y devuelve la primera coincidencia. De ahí salen directamente las reglas de visibilidad de Nova: una declaración interior tapa a una exterior, que es lo que detecta la advertencia W002, y un nombre que no aparece en ningún ámbito da el error E301.

El análisis semántico recorre el AST en **dos pasadas**. La primera recolecta solo las firmas globales -módulos importados, constantes y funciones-; la segunda entra a los cuerpos. Esa separación es la que permite que una función llame a otra declarada más abajo sin obligar a declaraciones adelantadas.

> **El detalle que conviene explicar:** además de la pila, la tabla mantiene un **listado histórico** (`Entrada`) que sobrevive al cierre de los ámbitos. La pila se vacía al terminar el análisis, que es su naturaleza, pero `--sym` tiene que imprimir todos los símbolos del programa al final, y la fase 4 necesita volver a consultar los desplazamientos de locales cuyo ámbito ya se cerró. Sin ese listado, la información existiría durante el análisis y desaparecería justo cuando hace falta.

Cada símbolo guarda su clase (variable, parámetro, constante, función, módulo), su tipo y dimensiones, si es global, su desplazamiento o dirección, su tamaño en bytes y la línea donde se declaró. Las funciones guardan además su firma completa y el tamaño del marco.

QUÉ	ARCHIVO	LÍNEAS
Estructura **Symbol**: todo lo que se sabe de un nombre	`fase3_semantico/symtab.h`	`63-106`
Pila de ámbitos **Scope**	`fase3_semantico/symtab.h`	`110-117`
Listado histórico **Entrada** (sobrevive al cierre)	`fase3_semantico/symtab.h`	`119-150`
Estado global **SymbolTable**	`fase3_semantico/symtab.h`	`152-162`
Apilar y desapilar ámbito	`fase3_semantico/symtab.c`	`236-258`
Búsqueda hacia afuera (3 variantes)	`fase3_semantico/symtab.c`	`280-305`
Inserción de variable, parámetro y función	`fase3_semantico/symtab.c`	`338-428`
Volcado que imprime `--sym`	`fase3_semantico/symtab.c`	`505-576`
Las dos pasadas del análisis semántico	`fase3_semantico/semantic.c`	`1394-1420`

2. Asignación de direcciones de memoria

El mapa de memoria no lo inventa el compilador: lo toma del ISA de CERBERO y lo declara como constantes, de modo que si la arquitectura cambia hay un solo lugar que tocar.

```
0x0000-0x3FFF   globales      (16 KB, los asigna este compilador)
0x4000-0xBFFF   buffers de E/S
0xC000-0xEFFF   heap reservado
0xF000-0xFFFF   pila, crece hacia abajo desde 0xFFFC
```

Hay dos regímenes distintos.

Globales. En Nova solo pueden ser `constans`; las variables globales mutables quedaron fuera del lenguaje a propósito, para que el modelo de memoria sea predecible. Se asignan con un contador corrido desde 0x0000: cada símbolo toma el valor actual de `global_memory_offset` y lo avanza por su tamaño. Si se pasa de 0x4000 se reporta desbordamiento del segmento.

Locales y parámetros. No tienen dirección fija: viven en `R14 + desplazamiento`, y R14 cambia en cada llamada. El desplazamiento se asigna dentro del marco, y al cerrar un ámbito se **recicla**: `local_memory_offset` vuelve a su valor previo mientras `max_local_offset` conserva la marca de agua, que es lo que de verdad se reserva. Así dos bloques hermanos comparten el mismo espacio de pila en vez de sumarse.

> **La pregunta que van a hacer:** si una local no tiene dirección fija, ¿qué es el `DMEM @0xFFE4` que muestra la tabla? Es la dirección de la **primera activación** de esa función. Se calcula propagando desde `principale` por el grafo de llamadas: la pila arranca en 0xFFFC y cada función baja su marco. Es exacta para esa llamada y se puede comprobar contra un volcado de memoria del procesador; en una llamada anidada -o en cada vuelta de una recursión- la misma variable queda un marco más abajo. Por eso la tabla muestra las dos cosas: la dirección absoluta y el `= r14+M`, que es el desplazamiento que realmente codifica la instrucción.

QUÉ	ARCHIVO	LÍNEAS
Mapa de memoria de CERBERO (constantes)	`fase3_semantico/symtab.h`	`24-28`
Contador de globales y chequeo de desborde	`fase3_semantico/symtab.c`	`322-331`
Desplazamiento de locales dentro del marco	`fase3_semantico/symtab.c`	`332-338`
Reciclaje de offsets y marca de agua al cerrar ámbito	`fase3_semantico/symtab.c`	`245-262`
Medida del marco de cada función	`fase4_codigo/codegen.c`	`544-607`
Propagación de bases por el grafo de llamadas	`fase4_codigo/codegen.c`	`1733-1761`
Layout completo: offsets y direcciones a la tabla	`fase4_codigo/codegen.c`	`1762-1800`
Registro de la dirección en el símbolo	`fase3_semantico/symtab.c`	`429-504`

3. Realización de saltos y resolución de referencias

El camino de un salto tiene **tres etapas**, y conviene contarlas en ese orden porque cada una resuelve un problema distinto.

3.1 Generación: etiquetas simbólicas

En el código de tres direcciones los saltos son simbólicos. `tac_new_label()` entrega etiquetas frescas y cada estructura de control las usa según su patrón. El ciclo `dum`, por ejemplo:

```
L_ini:
    <condición>
    ifFalse <cond> goto L_fin
    <cuerpo>
    goto L_ini
L_fin:
```

Las dos etiquetas se apilan además en una pila de ciclos, para que `rumpe` y `perge` sepan a cuál saltar: `rumpe` va a L_fin y `perge` a L_ini. Esa pila es también la que permite detectar el error E310, cuando alguna de las dos aparece fuera de un ciclo.

3.2 Registro: a qué bundle cae cada etiqueta

Al construir el programa, `registrar_etiqueta()` anota en qué bundle cae cada etiqueta. Como un bundle de CERBERO ocupa 16 bytes -128 bits, cinco slots-, la dirección de una etiqueta es simplemente su índice de bundle por 16. Después `resolver_direcciones()` recorre todos los slots buscando referencias de la forma

`&etiqueta` y las sustituye por ese número.

3.3 Relajación: el salto que no alcanza

Este es el punto fuerte del tema. El desplazamiento de un salto puede **no caber** en el inmediato de la instrucción, y eso solo se sabe después de haber ubicado todo el programa. `construir_programa()` lo resuelve iterando: arma el programa, `marcar_desbordados()` señala los saltos que no alcanzan, esos se reescriben en **forma larga** -cargar la dirección en un registro y saltar de forma indirecta- y se vuelve a armar. Como la forma larga ocupa más, puede desbordar otros saltos que antes sí cabían, así que el proceso se repite **hasta que converge**, con un tope de 8 pasadas. Eso es relajación de saltos, el mismo problema que resuelve cualquier ensamblador real.

El resultado se expone en dos lugares: la tabla de etiquetas con sus direcciones queda como comentario dentro del `.asm`, y aparece también bajo la tabla de símbolos en `novac --sym`, que es donde el profesor la buscó en la revisión anterior.

QUÉ	ARCHIVO	LÍNEAS
Etiquetas y temporales frescos	`fase4_codigo/tac.c`	`70-84`
Patrón del condicional `si`	`fase4_codigo/tac.c`	`762-786`
Patrón del ciclo `dum` y pila de ciclos	`fase4_codigo/tac.c`	`787-813`
Patrón del ciclo `per`	`fase4_codigo/tac.c`	`814-856`
Registro de etiqueta a bundle	`fase4_codigo/codegen.c`	`235-247`
Sustitución de `&etq` por dirección	`fase4_codigo/codegen.c`	`1635-1655`
Detección de saltos desbordados	`fase4_codigo/codegen.c`	`1609-1634`
Bucle de relajación hasta converger	`fase4_codigo/codegen.c`	`1656-1670`
Tabla de etiquetas en el `.asm`	`fase4_codigo/codegen.c`	`1335-1360`
Tabla de etiquetas expuesta a `--sym`	`fase4_codigo/codegen.c`	`282-310`
Impresión bajo la tabla de símbolos	`main.c`	`549-578`

4. Iteración 1: el IDE

Cómo está hecho, por qué de esa forma, y dos preguntas concretas.

4.1 La decisión de arquitectura

La restricción del curso era que la lógica del IDE fuera en C y la interfaz pudiera ir en Python. La forma en que se cumple es más fuerte que eso: **novaide enlaza las mismas unidades objeto que novac**, no una copia ni una reimplementación. En el Makefile, `NUCLEO_OBJS` es la lista de objetos del compilador sin `main.o`, y el binario del IDE se enlaza contra ella.

La consecuencia: el editor **no puede** discrepar del compilador. Los errores que subraya son los que emiten `nova.l`, `nova.y` y `semantic.c`, con la línea, la columna y el largo que esas fases calcularon. La tabla de símbolos es la de `symtab.c`, con las direcciones que calcula la fase 4. Y hasta el coloreado del código se pinta con el flujo de tokens de `yylex()`: si mañana se agrega una palabra reservada a `nova.l`, el editor la pinta como reservada sin tocar una línea de JavaScript.

El servidor en Python es **solo transporte**: no contiene una sola regla del lenguaje. Escribe el buffer del editor en un archivo temporal, invoca a `novaide` y devuelve su JSON tal cual.

`novaide` atiende **un trabajo por ejecución** y termina. La razón es concreta: el lexer y el parser que generan flex y bison llevan estado global -`yylval`, `yylloc`, el buffer de entrada, la pila de llaves abiertas, los contadores de diagnósticos-, y reutilizarlo entre análisis dentro de un mismo proceso es una fuente conocida de resultados que

dependen del orden de las peticiones. Arrancar un proceso cuesta menos de un milisegundo y, a cambio, cada análisis parte del mismo estado que una compilación recién lanzada.

Para que esto fuera posible hubo que tocar `diag.c`: se le agregó un **modo captura** que guarda cada diagnóstico como registro estructurado -con línea, columna y largo- en vez de escribirlo en stderr, y que no aborta al llegar al máximo de errores. El compilador de línea de comandos nunca enciende ese modo, así que su comportamiento no cambió.

4.2 Cómo se filtran los archivos del panel izquierdo

El filtrado ocurre en **dos etapas**, y la división es deliberada.

Etapa 1, en el servidor. `listar_fuentes()` recorre el árbol del proyecto con `os.walk`, poda los directorios `.git`, `ARQUITECTURA`, `__pycache__` y `entregables` y todo lo que empiece por punto, se queda solo con los archivos `.nv`, devuelve rutas relativas ordenadas y corta a 500 para que un árbol grande no cuelgue la interfaz.

Etapa 2, en el navegador. `cargarListaArchivos()` filtra por subcadena, sin distinguir mayúsculas, sobre esa lista ya en memoria.

El filtro fino es del lado del cliente a propósito: la lista completa ya viajó una vez, así que filtrar mientras se escribe no necesita otro viaje al servidor. Y la poda es del lado del servidor porque es ahí donde está el disco. Hay además una comprobación de seguridad: `ruta_segura()` impide que una petición lea fuera del árbol del proyecto.

4.3 ¿Se guarda el ensamblador al darle Ejecutar?

> **Respuesta directa: no, el archivo no se guarda** -todavía-. El **texto** del ensamblador sí aparece completo en el panel de Salida, porque `novac -s` lo escribe por stdout. Pero los archivos `.asm` y `.nvx` se generan junto a una copia **temporal** del buffer y el servidor los borra en cuanto mide su tamaño, que es lo que se reporta como *binario: N bytes*.

Por qué todavía no, con tres razones reales:

1. El IDE compila el **buffer del editor**, no el archivo en disco. Ese buffer puede tener cambios sin guardar, o un nombre que todavía no existe como archivo. «Dejar el .asm junto al fuente» no está definido si el fuente aún no existe.

2. Escribir artefactos en el árbol del proyecto en cada clic lo ensuciaría: compilar seis veces dejaría seis pares de archivos sueltos.

3. El directorio temporal se borra al cerrar el servidor, así que aunque no se borraran los archivos no sobrevivirían a la sesión.

Lo que falta para cerrarlo es trabajo de interfaz, no de compilador: un botón «Guardar salida» que escriba el contenido del panel donde el usuario indique, y -cuando el buffer sí corresponda a un archivo ya guardado- dejar el `.asm` al lado. Desde la terminal eso ya funciona: `./novac -s programa.nv -o programa.nvx` deja el `.asm` donde corresponde.

4.4 Las cuatro capacidades que pide la especificación

Sugerencias de código en dos capas: una léxica que siempre funciona -un archivo a medio escribir tokeniza aunque no parsee- y una semántica que agrega tipo y dirección cuando el archivo analiza. El contexto decide qué se ofrece: tras `:` o `->` solo los cuatro tipos; fuera de las funciones solo `constans` y `functio`, porque ofrecer `varia` ahí sería sugerir el error E320.

Corrección automática con dos fuentes: el propio parser, que ya clasifica sus errores y sabe exactamente qué token esperaba, y un balanceo de delimitadores que ve el archivo entero. Para la llave que falta se usa la **sangría** para decidir dónde va: ponerla al final del archivo también equilibra las llaves, pero deja el código dentro del bloque equivocado y el programa compila haciendo otra cosa.

Errores marcados gráficamente sobre el rango exacto que reporta el compilador, con un catálogo de los 54 códigos que agrega lo que el mensaje no dice: por qué el lenguaje lo prohíbe y cuál es el cambio que lo arregla.

Detección de clones por flujo de tokens, con la función como unidad. Tipo I: secuencias idénticas. Tipo II: idénticas tras normalizar nombres y literales, comprobando que el renombrado sea **consistente** -una biyección entre identificadores, que es la condición de *parameterized match* de Baker-. Tipo III: similitud alta medida con la subsecuencia común más larga sobre una codificación parametrizada, más una racha contigua mínima.

QUÉ	ARCHIVO	LÍNEAS
Enlace compartido novac / novaide	`Makefile`	`32-58`
Modo captura de diagnósticos	`fase1_lexico/diag.h`	`7-33`
Protocolo y trabajos del núcleo	`fase5_ide/novaide.c`	`111-170`
Diagnósticos, símbolos y etiquetas	`fase5_ide/ide_diag.c`	`162-297`
Sugerencias de código	`fase5_ide/ide_completar.c`	`625 líneas`
Corrección automática	`fase5_ide/ide_arreglar.c`	`546 líneas`
Detección de clones	`fase5_ide/ide_clones.c`	`557 líneas`
Catálogo de los 54 diagnósticos	`fase5_ide/ide_catalogo.c`	`343 líneas`
Listado y poda de archivos	`ide/servidor.py`	`222-234`
Filtro por subcadena en el navegador	`ide/web/app.js`	`1151-1163`
Comprobación de ruta segura	`ide/servidor.py`	`187-196`
Compilar: banderas, -o y borrado del temporal	`ide/servidor.py`	`76-135`

5. Iteración 2: código intermedio y bloques básicos

5.1 Cómo se genera el TAC

El código de tres direcciones se genera recorriendo el AST ya validado por la fase 3. Opera sobre una máquina abstracta de **registros virtuales ilimitados** y es **independiente de CERBERO**: quien lo baja a bundles es `codegen.c`. Esa separación es la que permite que las optimizaciones de la Iteración 3 trabajen sobre el IR sin saber nada del procesador.

Cada instrucción tiene a lo sumo **un destino y dos operandos**, más un entero auxiliar para lo que no es un nombre: la posición de un argumento, la dimensión de `longitudo`, la palabra de una llave. Las expresiones se descomponen con temporales frescos (`t0, t1, ...`) y las estructuras de control con etiquetas frescas, según los patrones de la sección 3.

5.2 Identificación de bloques básicos

Un bloque básico es una secuencia máxima de instrucciones donde el flujo **entra por la primera y sale por la última**: no hay saltos hacia adentro salvo al inicio, ni hacia afuera salvo al final. El algoritmo es el clásico de **líderes**, en dos pasadas por función.

Una instrucción es líder si cumple alguna de tres condiciones:

```
a)  es la primera de la función
b)  lleva una etiqueta  (puede ser destino de un salto)
c)  sigue inmediatamente a un salto, un retorno o un halt
```

La segunda pasada delimita: cada líder abre un bloque que llega hasta la instrucción anterior al siguiente líder.

> **Dos decisiones que hay que poder justificar.** `TAC_CALL` **no** parte el bloque: el grafo es intraprocedural y una llamada devuelve el control a la instrucción siguiente, así que dentro de la función es una instrucción más. Partir en cada llamada daría un grafo más fragmentado sin información adicional, y achicaría los bloques justo donde la calendarización en bundles necesita margen. Y **toda etiqueta abre bloque**, aunque nadie salte a ella: comprobar antes si está referenciada ahorraría algún bloque, pero el grafo quedaría dependiendo del orden de recorrido, y el bloque de más se reporta como inalcanzable, que es información útil y no un defecto.

5.3 Cómo se genera el grafo

Las aristas de salida de un bloque dependen de **cómo terminan**:

TERMINA EN	ARISTAS DE SALIDA
caída	al bloque siguiente
salto — goto L	al bloque de L
condicional — ifFalse/ifTrue L	**dos**: a L y al siguiente (la caída)
retorno — ret / ret v	ninguna: es salida de la función
halt	ninguna
tailcall — @cauda	de regreso al bloque 0

Los **predecesores se derivan de los sucesores**, de modo que no pueden quedar descuadrados; hay una prueba automática que lo verifica. Un recorrido en profundidad desde el bloque de entrada marca la **alcanzabilidad**: los bloques que no se alcanzan son código muerto. Y una arista hacia un bloque de índice menor o igual es una **arista de regreso**, lo que identifica los lazos: entre el destino y el origen está el cuerpo del ciclo.

Medido sobre los 23 programas de prueba del proyecto, el grafo da **354 bloques**, de los cuales **16 son inalcanzables** -el `ret` de seguridad que el generador agrega al final de cada función, y saltos muertos- y **25 están vacíos**, con solo su etiqueta. Esos dos números son la línea base medible contra la que se evaluará la eliminación de código muerto de la Iteración 3.

QUÉ	ARCHIVO	LÍNEAS
Repertorio de operaciones y estructuras del TAC	`fase4_codigo/tac.h`	`23-124`
Generación del TAC desde el AST	`fase4_codigo/tac.c`	`940-1010`
Expresiones con temporales frescos	`fase4_codigo/tac.c`	`608-708`
Sentencias y estructuras de control	`fase4_codigo/tac.c`	`709-939`
Formateador único de instrucción (--ir, --bloques, IDE)	`fase4_codigo/tac.c`	`991-1060`
Estructuras **Bloque** y **GrafoFlujo**	`fase6_optimizacion/bloques.h`	`30-69`
Qué instrucciones cierran un bloque	`fase6_optimizacion/bloques.c`	`67-80`
Pasada 1: contar líderes	`fase6_optimizacion/bloques.c`	`132-146`
Pasada 2: delimitar bloques	`fase6_optimizacion/bloques.c`	`147-169`
Tendido de aristas según el tipo de fin	`fase6_optimizacion/bloques.c`	`170-211`
Alcanzabilidad por recorrido en profundidad	`fase6_optimizacion/bloques.c`	`212-233`
Detección de lazos por aristas de regreso	`fase6_optimizacion/bloques.c`	`234-256`
Construcción del grafo completo	`fase6_optimizacion/bloques.c`	`257-280`
Serialización del grafo para el IDE	`fase5_ide/ide_bloques.c`	`87-164`
Bandera `--bloques`	`main.c`	`634-660`

6. Iteración 3: vivacidad y código muerto

De las cuatro optimizaciones que pide la iteración, las dos que habilitan a las otras dos.

Una variable está **viva** en un punto del programa si el valor que tiene ahí **puede llegar a leerse** antes de que alguien lo sobrescriba. Si no está viva, el valor es basura y la instrucción que lo calculó sobra. Lo importante de

la definición es el «puede»: basta que exista **un** camino que lo lea, y por eso no se puede decidir mirando el programa línea por línea: hay que recorrer el grafo.

6.1 Por qué el análisis va hacia atrás

Lo que decide si un valor sirve no es lo que pasa **antes** de calcularlo sino lo que pasa **después**. La información viaja entonces desde los usos hacia las definiciones, en contra de las aristas. Por cada bloque se calculan cuatro conjuntos:

```
USO[B]       nombres que B lee ANTES de escribirlos
DEF[B]       nombres que B escribe antes de leerlos (los mata)
VIVO_SAL[B] = union de VIVO_ENT[S] para cada sucesor S de B
VIVO_ENT[B] = USO[B] union (VIVO_SAL[B] menos DEF[B])
```

USO y DEF se calculan recorriendo el bloque **hacia adelante** una sola vez, y el orden importa: en `a = a + 1` la lectura ocurre antes que la escritura, así que `a` pertenece a USO y **no** a DEF.

6.2 El punto fijo

> **Las dos últimas ecuaciones son mutuamente recursivas** -- en un lazo un bloque es sucesor de sí mismo por transitividad -- así que no se resuelven de una pasada. Se **itera**: todos los conjuntos arrancan vacíos y se recorre el grafo hasta que una pasada completa no cambia nada. **Termina** porque los conjuntos solo crecen y el universo de nombres es finito: una sucesión creciente y acotada se estanca. Y es el punto fijo **menor**, porque se parte de vacío: no se da por vivo nada que no se haya demostrado vivo. Los bloques se recorren en **orden inverso** para que cada pasada aproveche lo ya calculado de los sucesores; eso no cambia el resultado, solo el costo.

En la práctica: 1 pasada sin saltos, 2 con un condicional, 3 con un lazo, 3 o 4 con lazos anidados. `novac --vivacidad` reporta el número real por función, así que no hay que creerlo: se mira.

6.3 La condición de frontera

A la salida de un bloque que retorna, el marco de la función deja de existir: no queda vivo ningún local ni ningún temporal. **Esa condición es la que arranca todo** -- sin ella, los conjuntos vacíos iniciales serían también el punto fijo y nada estaría vivo nunca. Lo único que sobrevive son los nombres que **escapan** por `&` y los globales, porque su valor se puede observar desde fuera.

6.4 Qué se elimina

Bloques inalcanzables, que ya los marcaba el grafo de la Iteración 2. **Definiciones muertas**, por recorrido hacia atrás dentro del bloque partiendo de VIVO_SAL. Y el **destino de una llamada que nadie lee**: la llamada se queda, el destino no. Las tres se repiten hasta el punto fijo, porque cada una habilita a las otras, y entre ronda y ronda **el grafo se reconstruye**, de modo que la vivacidad se calcula siempre sobre el programa que de verdad quedó.

Al marcar una instrucción como sobrante **no se avivan sus lecturas**, porque va a desaparecer y no va a leer nada. Eso hace que una cadena entera de cálculos muertos caiga en **una sola pasada** en vez de necesitar una ronda por eslabón. Hay una prueba que lo fija, porque si el recorrido estuviera mal el síntoma sería justamente ese.

6.5 Qué NO se elimina, y por qué

> **Esta es la parte que hay que poder defender:** una eliminación de más cambia el programa, y en un compilador ese es el peor error posible. Sobre `test/ide/demo_iteracion3.nv`, donde tres asignaciones tienen el destino muerto, se ve exactamente qué sobrevive y qué no.

```
functio intocables:
    t13 = #1000
    t14 = t13 / d        <- SE QUEDA: aborta con 136 si d es cero
    t15 = longitudo v, dim 0
    check d, t15         <- SE QUEDA: aborta con 134 fuera de rango
                         <- 't16 = v[d]' SE FUE: leer memoria es puro
    call con_sobras, 2   <- SE QUEDA, pero sin destino: nadie lo lee
```

```
    t20 = #0
    ret t20
```

Quitar el `v[d]` y conservar el `check` que estaba una línea antes sale de tener la semántica de cada operación en **una sola tabla**, `viv_efecto()`, que comparten el análisis, la eliminación y la vista del IDE. Tampoco se tocan las escrituras a memoria -- haría falta análisis de alias --, los nombres que escapan por `&`, los globales, ni los bloques vacíos que solo llevan etiqueta, que son destino de saltos.

6.6 La advertencia W010, y por qué no sobra con W001

Cuando el análisis encuentra una asignación muerta a una variable del programa, el compilador lo dice, y el editor la subraya **mientras se escribe**: es la primera marca del IDE que sale de un análisis de flujo de datos sobre el grafo, y no de la sintaxis.

W001 pregunta *¿alguien menciona este nombre?* (fase 3, sobre el árbol). W010 pregunta *¿el valor que se guarda aquí llega a leerse?* (fase 6, sobre el grafo). Que son distintas se ve en el mismo archivo de demostración, donde aparecen las tres combinaciones:

VARIABLE	W001	W010	POR QUÉ
`sobra_1`	—	**sí**	**Sí se menciona**: la línea siguiente la lee. Pero su valor igual está muerto, porque la cadena entera lo está.
`division`	sí	sí	Ni se menciona ni se lee: las dos tienen razón.
`inalcanzable`	**sí**	—	Vive en un bloque inalcanzable; no se duplica, W003 ya avisó.

`sobra_1` es el ejemplo que conviene tener a mano: es justo el caso que una revisión a ojo no encuentra, porque la variable *se usa* y parece viva. Hay que seguir la cadena hasta el final y descubrir que nadie lee el último eslabón.

6.7 Las métricas, y el intercambio

Hay **dos clases de métrica y se miden en lugares distintos**. El compilador mide lo que puede contar exactamente: cuánto tardó, cuántas instrucciones quedaron, cuántos bundles salieron y cuántas ranuras llevan trabajo. **No estima tiempo de ejecución**: eso lo dan los contadores del procesador al correr el binario sobre el RTL. Sobre el archivo de demostración:

	-O0	-O1
tiempo de compilación	`0.95 ms`	**1.58 ms**
del cual, la fase 6	–	0.30 ms
instrucciones de tres direcciones	`80`	**65** (-18.8%)
bundles emitidos	`585`	**516** (-69)
bytes de IMEM	`9 360`	**8 256**
ocupación media	`0.41 de 5`	0.40 de 5
bundles totalmente vacíos	`345 (59.0%)`	312 (60.5%)

El compilador tarda más para que el programa tarde menos. Cada bundle es un ciclo de emisión, así que son 69 ciclos menos cada vez que el programa corre.

6.8 El resultado honesto, y lo que sigue

Sobre los 23 programas del corpus la reducción es de **0.8% en instrucciones y 0.27% en bundles**, y en los Test1 a Test6 del profesor la eliminación **no encuentra nada**. El código escrito a mano y limpio casi no tiene código muerto, y hay que decirlo así. La pasada sirve por otras tres razones: es el **requisito previo** del spilling, que necesita los rangos de vida; es la **limpieza** que corre después del desenrollado, que sí genera código muerto; y como **diagnóstico** ya encontró dos asignaciones muertas reales en nuestro propio `matriz_1000.nv`, que estaban ahí desde el Proyecto I. El número que importa para la Iteración 4 está en la

misma tabla: **ocupación de 0.40 ranuras de 5**. El VLIW se está usando como un procesador escalar caro, y eso lo arregla la calendarización.

6.9 Cómo se verificó que no cambia el programa

Dos comprobaciones, las dos contra el proyecto de arquitectura, y las dos aceptan `OPT=-O1`:

```
OPT=-O1 sh tools/verificar.sh 12   -> 20 identicos bit a bit, 0 distintos
make prueba-rtl-optimizado         -> los 12 programas, RESULTADO: OK
```

La primera pasa nuestro ensamblador optimizado por el **ensamblador independiente del equipo de CE4301** y compara bit a bit: el código que sale de la fase 6 sigue siendo CERBERO válido. La segunda es la de verdad: **la misma batería, los mismos valores de retorno esperados, compilando con -O1**, corriendo sobre el RTL del procesador. Si una sola cambiara de resultado, la fase 6 estaría rota. Además, 233 + 101 pruebas y todo el corpus bajo valgrind sin un solo error.

Y hay dos confirmaciones más que regala la propia tabla de `--sym`, por caminos distintos de todo lo anterior:

1. Que la división sobrevivió se ve sin mirar el código intermedio. La rutina `__nova_div32` y sus ocho etiquetas `__dv_*` **solo se emiten si el programa tiene alguna división**: la fase 4 enciende `usa_div` al encontrar un TAC_DIV. Si la eliminación hubiera borrado la división muerta, esas nueve etiquetas desaparecerían de la tabla. Siguen ahí con -O1.

2. Los 69 bundles ahorrados cuadran por dos caminos. La última etiqueta, `__scribe_int`, pasa del bundle 469 al 400: 69 menos. Ese número sale de la **relajación de saltos**, que ubica cada etiqueta iterando hasta converger. El contador del **emisor** dice 585 - 516 = 69. Dos mecanismos distintos, el mismo ahorro; si no coincidieran, uno estaría describiendo un programa que no es.

QUÉ	ARCHIVO	LÍNEAS
Tabla de efectos: qué lee y escribe cada operación	`fase6_optimizacion/vivacidad.c`	`75-232`
Universo de nombres, escapes y globales	`fase6_optimizacion/vivacidad.c`	`259-320`
Cálculo de USO y DEF por bloque	`fase6_optimizacion/vivacidad.c`	`321-348`
Iteración hasta el punto fijo y condición de frontera	`fase6_optimizacion/vivacidad.c`	`349-418`
Recorrido hacia atrás: qué sobra en un bloque	`fase6_optimizacion/vivacidad.c`	`454-541`
Listado de `--vivacidad`	`fase6_optimizacion/vivacidad.c`	`561-656`
Marcado de lo que se elimina	`fase6_optimizacion/muerto.c`	`48-152`
Barrido: desenlazar sin romper el grafo	`fase6_optimizacion/muerto.c`	`153-177`
Rondas hasta el punto fijo	`fase6_optimizacion/muerto.c`	`178-232`
Banderas `-O0 / -O1`	`main.c`	`383-394`
La fase 6 en la tubería	`main.c`	`682-695`
Layout de `--sym` también optimizado	`main.c`	`564-581`
Reporte de `--metricas`	`main.c`	`853-926`
Estadísticas de bundles y ocupación	`fase4_codigo/codegen.c`	`317-334`
W010 en el editor, en cada pausa de tecleo	`fase5_ide/ide_diag.c`	`192-213`
Conjuntos y marcas para la vista del IDE	`fase5_ide/ide_bloques.c`	`50-145`
32 pruebas del bloque [7]	`test/ide/correr_pruebas.py`	`553-805`

7. Preguntas probables y respuesta corta

PREGUNTA	RESPUESTA
¿Por qué una local no tiene una sola dirección?	Porque vive en R14 + desplazamiento y R14 cambia en cada llamada. Lo que se muestra es la primera activación; el `= r14+M` es lo que codifica la instrucción.
¿Cómo saben que el IDE marca lo mismo que el compilador?	Porque enlaza las mismas unidades objeto. No hay un segundo analizador que pueda divergir. Hay pruebas que comparan las direcciones que reporta el IDE con las de `novac --sym`.
¿Qué pasa si un salto no alcanza?	Se reescribe en forma larga y se vuelve a armar el programa, iterando hasta converger. Es relajación de saltos.
¿Por qué una llamada no parte el bloque básico?	Porque el grafo es intraprocedural: la llamada devuelve a la instrucción siguiente. Que tenga efectos es una dependencia, no una arista.
¿Cómo se sabe que un bloque es código muerto?	No se alcanza desde el bloque de entrada en el recorrido en profundidad. Hoy hay 16 en los programas de prueba, y ninguno contiene cálculo.
¿Por qué el ensamblador no se guarda al compilar desde el IDE?	Porque se compila el buffer del editor sobre un archivo temporal. El texto sí queda en pantalla. Falta un botón de guardar, que es trabajo de interfaz.
¿Cómo se verifica que el binario es correcto?	Los 20 programas comparables salen **idénticos bit a bit** contra el ensamblador independiente del equipo de CE4301 (`tools/verificar.sh`), y los programas de ejecución se corren sobre el RTL del procesador comparando el valor de retorno.
¿Por qué el análisis de vivacidad va hacia atrás?	Porque lo que decide si un valor sirve no es lo que pasa antes de calcularlo sino lo que pasa después. La información viaja de los usos a las definiciones, en contra de las aristas.
¿Por qué hay que iterar, y por qué termina?	Porque en un lazo un bloque es sucesor de sí mismo por transitividad, así que las ecuaciones son recursivas. Termina porque los conjuntos solo crecen y el universo de nombres es finito. Se parte de vacío, así que es el punto fijo **menor**.
¿Por qué no se elimina una división muerta?	Porque `__nova_div32` termina el programa con el código 136 si el divisor es cero. Borrarla convertiría un programa que aborta en uno que no, y eso es observable. Lo mismo con `check`, que aborta con 134.
¿Y cómo saben que optimizar no cambia el programa?	La misma batería sobre el RTL del procesador, con los mismos valores de retorno esperados, compilando con -O1: `make prueba-rtl-optimizado`. Los 12 dieron OK.
Si casi no elimina nada, ¿para qué sirve?	Para tres cosas: es el requisito previo del spilling, que necesita los rangos de vida; es la limpieza que corre después del desenrollado, que sí genera código muerto; y como diagnóstico ya encontró dos asignaciones muertas reales en nuestro propio código del Proyecto I.
¿W001 y W010 no dicen lo mismo?	No. W001 pregunta si alguien **menciona** el nombre (fase 3, sobre el árbol); W010 si el valor que se guarda **llega a leerse** (fase 6, sobre el grafo). `sobra_1` lleva W010 y no W001: sí se menciona, pero su valor igual está muerto.

Comandos para tener a mano durante la revisión

```
./novac --sym prueba.nv          # símbolos con dirección + puntos de salto
./novac --ir  prueba.nv          # código de tres direcciones
./novac --bloques prueba.nv      # bloques básicos y grafo de flujo
./novac --vivacidad prueba.nv    # USO, DEF, VIVO_ENT, VIVO_SAL por bloque
./novac -s    prueba.nv -o p.nvx # ensamblador CERBERO (sí guarda el .asm)

F=test/ide/demo_iteracion3.nv    # el archivo de demostración de la Iter. 3
make vivacidad F=$F              # los conjuntos del análisis
make metricas  F=$F              # -O0 y -O1 lado a lado

make prueba                      # 233 pruebas del compilador
make prueba-ide                  # 101 pruebas del núcleo del IDE
sh tools/verificar.sh 12         # 20 programas idénticos bit a bit
OPT=-O1 sh tools/verificar.sh 12 # lo mismo, sobre el código optimizado
make prueba-rtl-optimizado       # la batería del RTL compilando con -O1
```